

MLDS Exploratory Data Analysis & Visualisation

Week 8 - Multivariate Data Visualisation

James S Martin

Spring Term, 2025

- The Wine Quality Dataset
- Repeated Univariate Visualisations
- Bivariate Data Visualisation
- Visualising pairwise dependence
- References

In this notebook, we will take a look at different visualisation techniques for multivariate data.

The Wine Quality Dataset

Through exercises, you will construct a number of visualisations that will allow an exploratory analysis of the popular 'Wine Quality' datasets. These data contain information on the physiochemical properties of both white and red variants of the Portuguese 'Vinho Verde' wine. These data are part of the University of California, Irvine (UCI) Machine Learning Repository. They are widely used to illustrate machine learning concepts, and were originally collected as part of a study that aimed to classify the quality of a wine based on its physiochemical properties (Cortez et al., 1998; more details available here (<http://www3.dsi.uminho.pt/pcortez/wine/>)). They will be particularly useful for our (EDAV) purposes, as the (two) datasets are multivariate, and comprise both categorical and numeric data variables.

We therefore start by loading the data. The data are contained in two separate CSV files, available at the UCI ML Repository: one for red varietals and one for white varietals. The following code cell reads the datasets directly from their respective webpages; if you are working in the Coursera RStudio environment, the CSV files have also been provided, and can alternatively be loaded in the normal way (e.g. using `read.csv()` or `readr::read_csv()`).

```
red.url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv"
red <- read.csv(red.url, header = TRUE, sep = ";")
red <- tibble(red)

white.url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-white.csv"
white <- read.csv(white.url, header = TRUE, sep = ";")
white <- tibble(white)
```

Exercise

1. Use `dplyr::bind_rows()` to combine the two datasets above into a single data frame (or tibble), named `wine`. Ensure that there is a new column created that identifies the dataset each observation came from.

```
wine <- bind_rows(list('red'=red,'white'=white), .id='wine.colour')
```

Repeated Univariate Visualisations

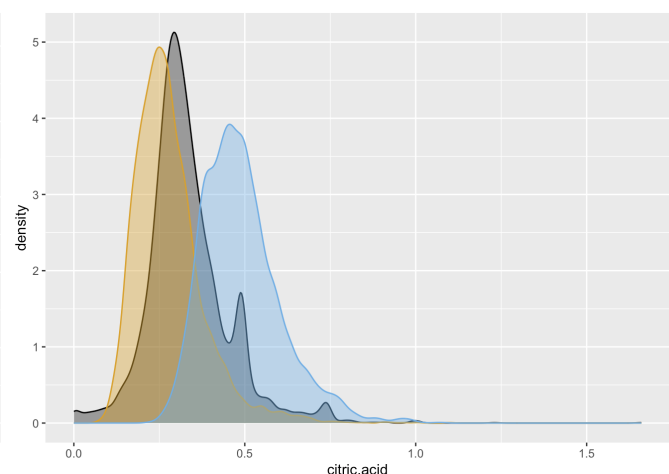
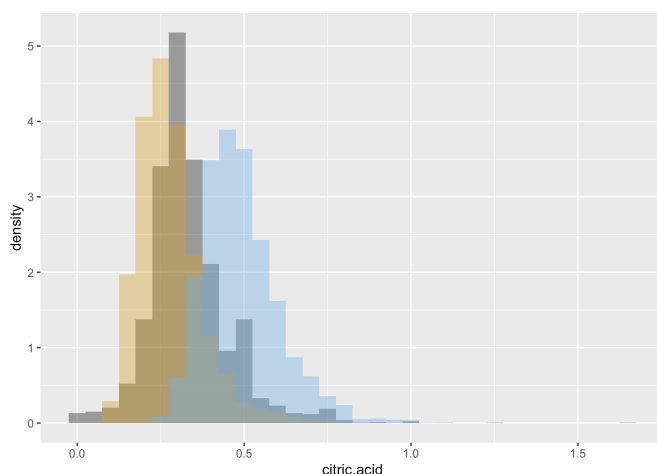
A straightforward first approach to visualising a multivariate dataset is to place multiple univariate visualisations in the same graphical object. This can be as straightforward as *overlaying* each variable's representation on the same axes. We have used this approach already in this module without much thought (e.g. when comparing histograms of zinc concentration levels around the Meuse river, before and after spatial declustering, in Week 6). It is perhaps worth casting a slightly more critical eye on this approach, however.

Suppose we wish to compare the univariate empirical distributions of citric acid, acetic acid (represented by `volatile.acidity`), and potassium sulphate (`sulphates`) in the white wines dataset. A 'vanilla' approach to this task might be to overlay the histogram (or possibly a smoothed estimate of the density) for each of the required variables.

```
cbPalette <- c("#000000", '#E69F00', '#56B4E9', '#009E73', '#F0E442', '#0072B2',
              '#D55E00', '#CC79A7')
# this is a colour-blind-friendly palette, obtained from http://www.cookbook-r.c
om/Graphs/Colors_(ggplot2)/

bwid <- 0.05
hists_overlaid <- ggplot(white) +
  geom_histogram(aes(x=citric.acid, y=after_stat(density)), binwidth=bwid, fill=cb
Palette[1], alpha=0.4) +
  geom_histogram(aes(x=volatile.acidity, y=after_stat(density)), binwidth=bwid, fi
ll=cbPalette[2], alpha=0.4) +
  geom_histogram(aes(x=sulphates, y=after_stat(density)), binwidth=bwid, fill=cbPa
lette[3], alpha=0.4)
hists_overlaid

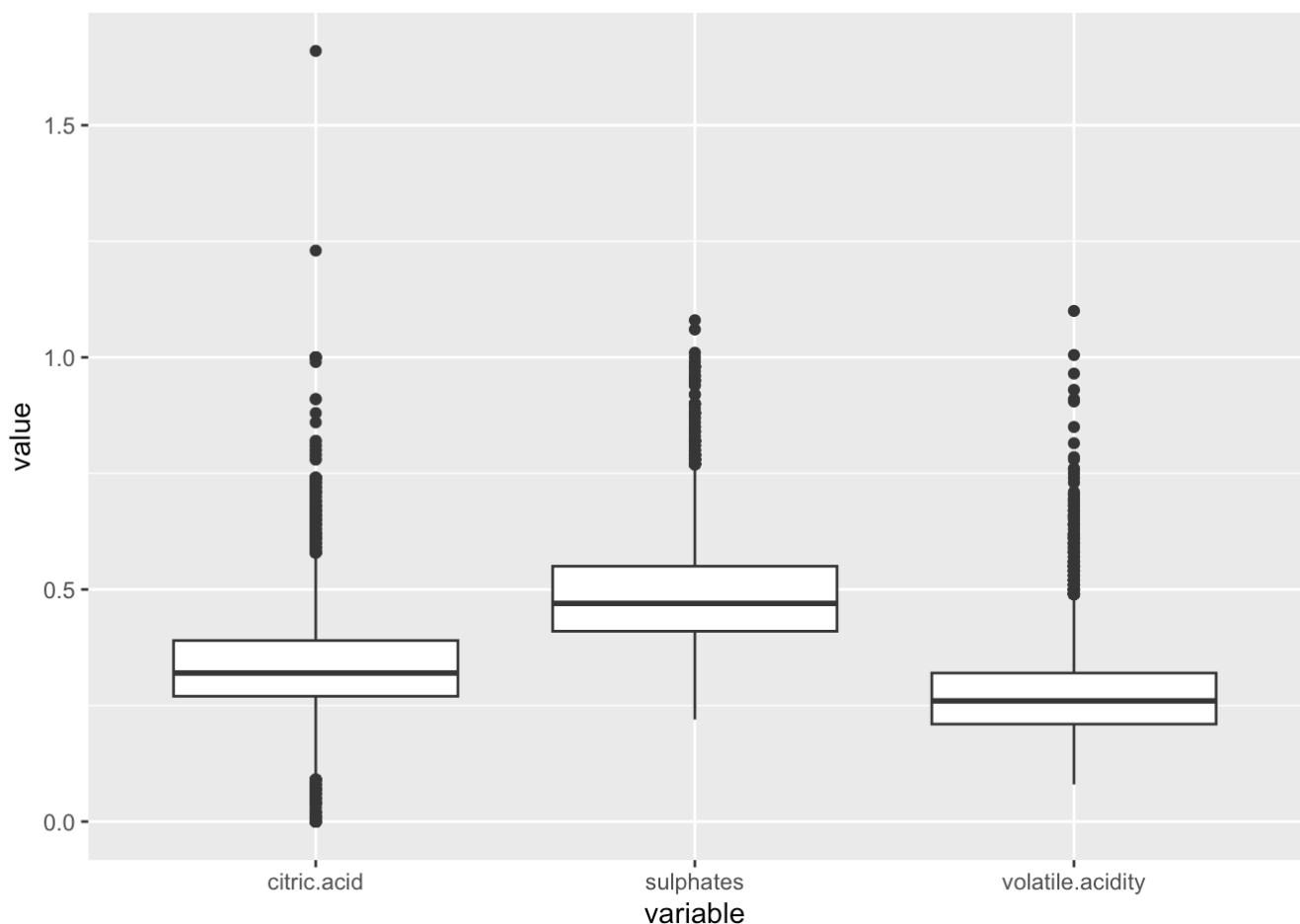
dens_overlaid <- ggplot(white) +
  geom_density(aes(x=citric.acid, y=after_stat(density)), fill=cbPalette[1], colou
r=cbPalette[1], alpha=0.4) +
  geom_density(aes(x=volatile.acidity, y=after_stat(density)), fill=cbPalette[2],
colour=cbPalette[2], alpha=0.4) +
  geom_density(aes(x=sulphates, y=after_stat(density)), fill=cbPalette[3], colour=
cbPalette[3], alpha=0.4)
dens_overlaid
```



By overlaying these univariate representations, we can quickly get a good idea of the shape of each variable's distribution, as well as a rough idea of the central tendency and spread of each distribution, and the degree to which these distributions overlap. On the other hand, these visualisations do not give a very clear comparison of the distributions' extremities; we cannot easily pick apart the overlapping tails, due to the low probability density in these regions. If this is our principal concern, it might be more beneficial for us to use boxplots, which allow us to concisely visualise the central tendency, spread and tails of a univariate distribution.

Suppose we wish to use boxplots to compare the univariate distributions of `citric.acid`, `volatile.acidity`, and `sulphates` in the white wines dataset. We will first reduce our dataframe to isolate these three variables, and then we can subsequently tidy this dataframe. This will essentially create a bivariate dataframe with the 'variable name' in the first column, and the corresponding numeric value in the second column. It is then straightforward to construct the required `ggplot()` object using `geom_boxplot()`.

```
white_tidy <- white %>%  
  select(c('citric.acid', 'volatile.acidity', 'sulphates')) %>%  
  pivot_longer(cols=everything(), names_to='variable')  
  
boxplots <- ggplot(data=white_tidy) +  
  geom_boxplot(aes(x=variable, y=value))  
boxplots
```

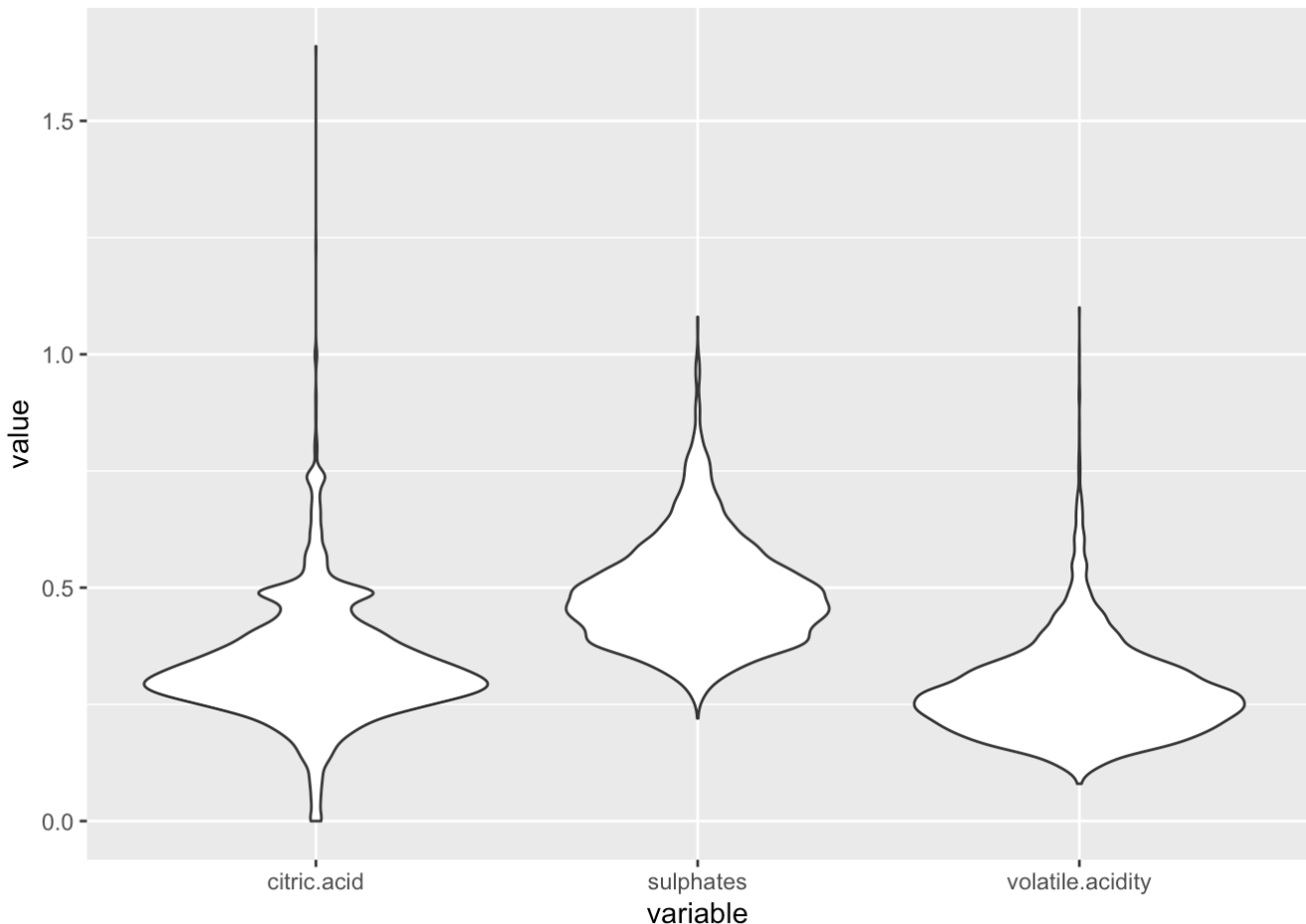


An alternative univariate visualisation that we might choose to make use of, is the violin plot. This is a compact representation, suitable for visualising continuous variables, which combines the compact display of the boxplot with the context of a density estimate.

Exercise

2. Use `geom_violin()` to create a lattice-style violin plot that displays the same variables as the boxplots above. Does this give you any additional information about the univariate distributions for these variables?

```
# Q2
violinplots <- ggplot(data=white_tidy) +
  geom_violin(aes(x=variable, y=value))
violinplots
```



In constructing the above demonstrations, the variables were carefully chosen such that their distributions covered a similar range. It may be beneficial to compare the univariate distributions of two variables that operate on different scales; we can visualise this effectively by adding a second axis to our plot.

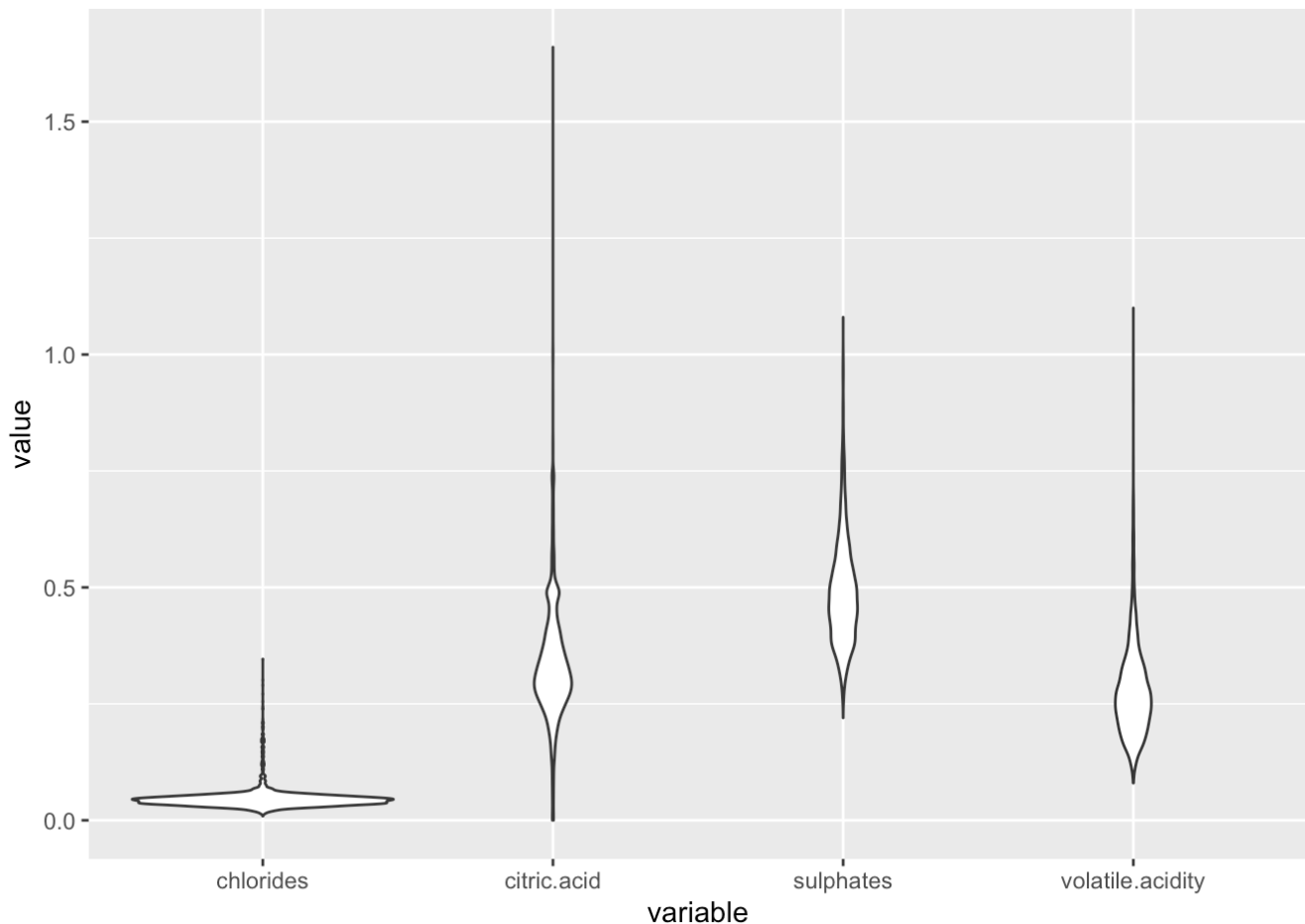
Exercises

Let us consider the three variables above, along with the additional variable `chlorides`.

3. Use the same approach as above to construct a tidy dataset `white_tidy2` containing these four variables.
4. Construct a lattice plot with either four boxplots or four violin plots.

```
# Q3
white_tidy2 <- white %>%
  select(c('citric.acid', 'volatile.acidity', 'sulphates', 'chlorides')) %>%
  pivot_longer(cols=everything(), names_to='variable')

# Q4
violinplots <- ggplot(data=white_tidy2) +
  geom_violin(aes(x=variable, y=value))
violinplots
```



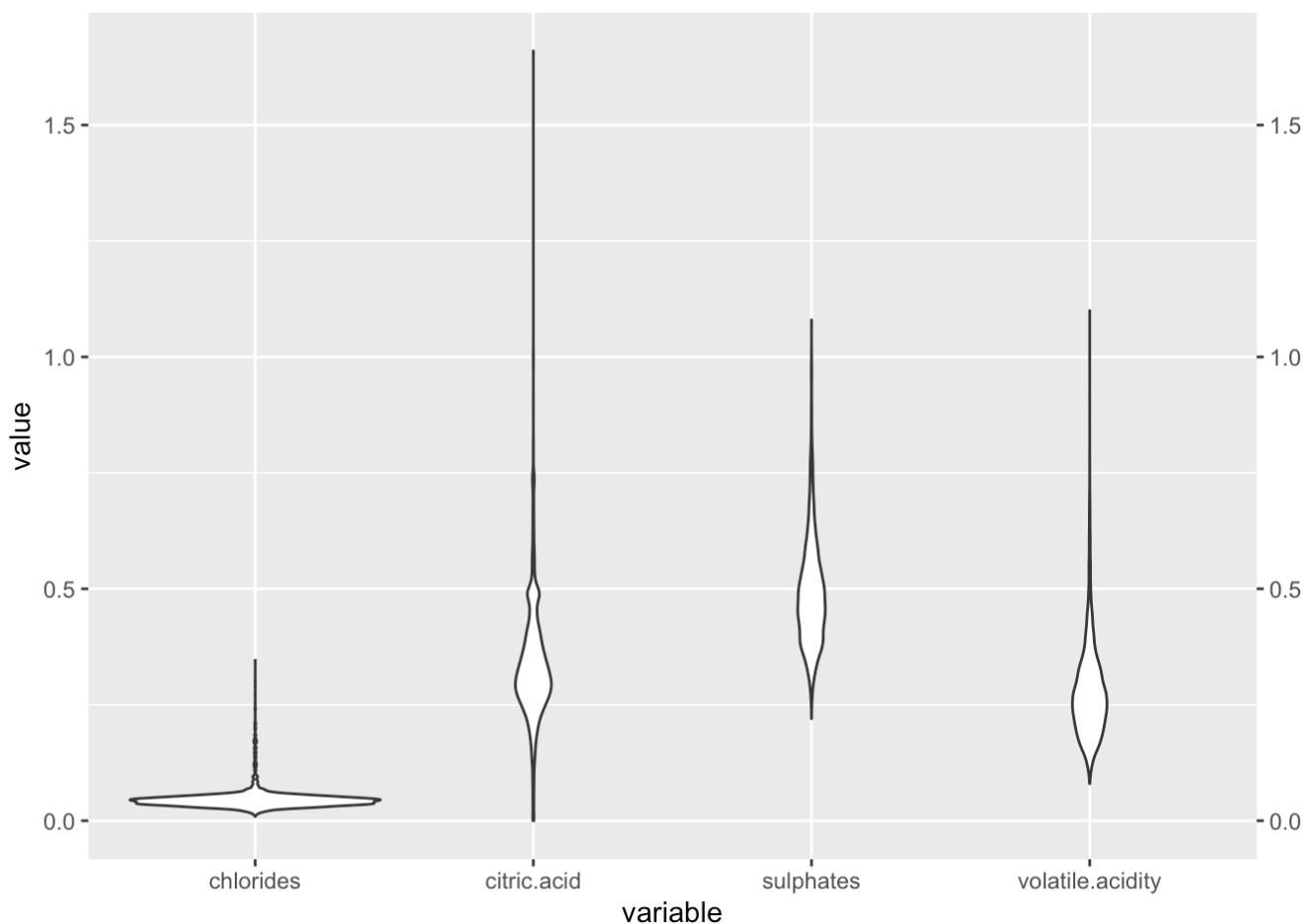
Adding the ggplot2 position scale `scale_y_continuous()` allows you to edit a number of properties of the vertical axis. The `sec.axis` argument allows us to initialise a secondary axis.

5. Use `scale_y_continuous(sec.axis=sec_axis(~.))` to duplicate the vertical axis on the opposite side of the plot.

A general aside: the ggplot functions `scale_x_continuous()` and `scale_y_continuous()` simply specify that the two plot axes are continuous. Replacing these with e.g. `scale*_log10()` will amend the scale of the axis accordingly. By default, when we map a continuous variable to the `y` aesthetic, `scale_y_continuous()` is silently applied by default; therefore calling this position scale without any arguments will change nothing in your plot. By calling it explicitly, however, we can modify properties of the corresponding axis.

`sec.axis=sec_axis(~.)` is used as an input argument to a position scale in order to create a secondary axis, opposite the axis referred to by the position scale. Here, the input `~.` (i.e. the first argument to `sec_axis()`) is an R formula that specifies the relationship between the primary and secondary axes.

```
# Q5
violinplots + scale_y_continuous(sec.axis=sec_axis(~.))
```



6. By consulting the 'help' documentation for `ggplot2::sec_axis()`, modify the `trans` argument to `sec_axis()` such that the secondary axis is a suitable range for representing the distribution of chlorides.

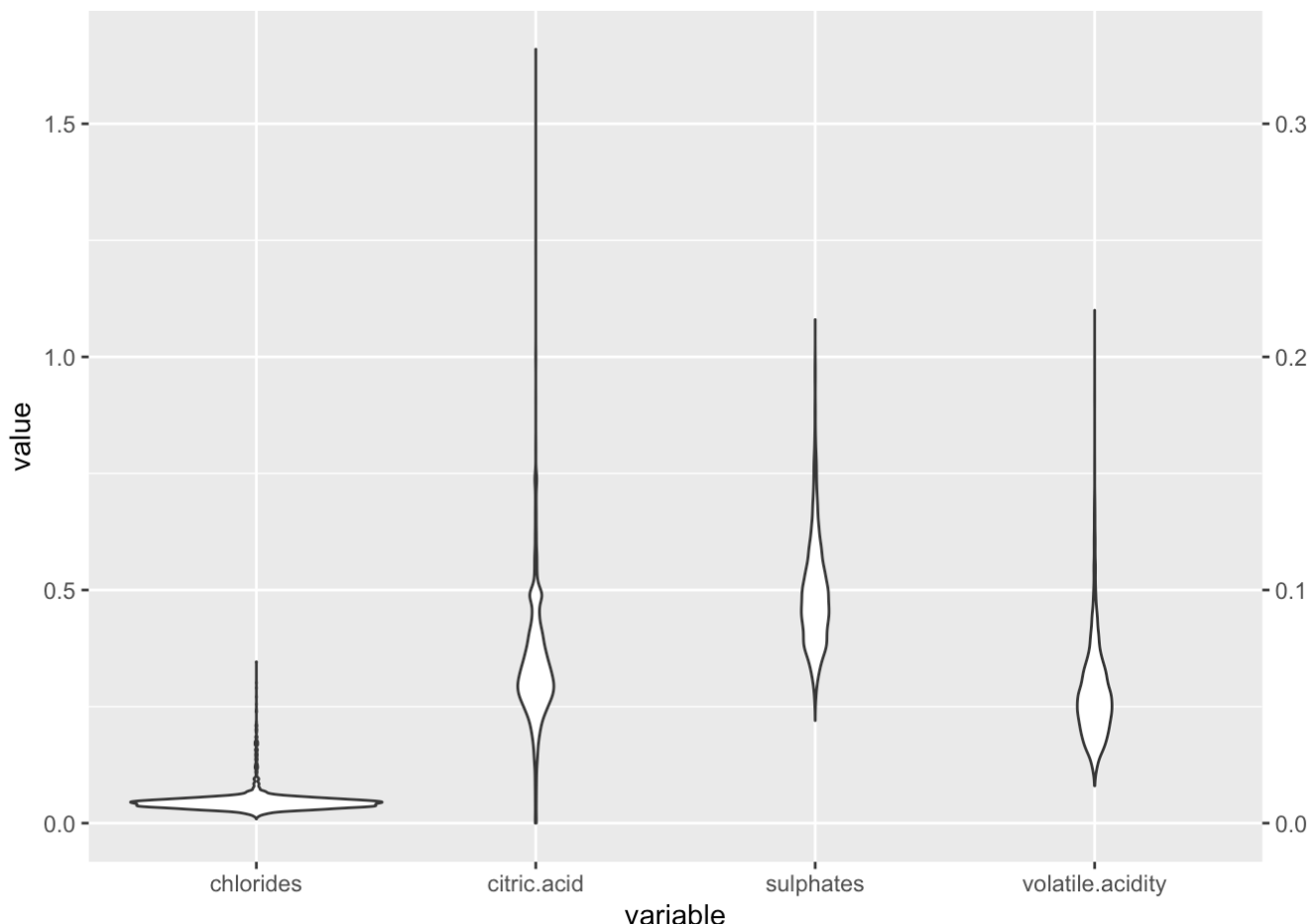
Above, we have initially given the `trans` argument as `trans = ~.`, specifying that the secondary axis has the same scale as the primary. We change this by modifying the formula `~.` – by using (e.g.) `trans = ~. * 2`, we could specify that the scale of the secondary axis is obtained by doubling the scale of the primary axis.

Using `summary()`, we seek a scaling factor that would bring the values of `chlorides` in line with those of the three variables already being considered (`citric.acid`, `volatile.acidity` and `sulphates`). We can see that, aligning by median value, a scaling factor between 6 and 11 would be appropriate, whereas aligning by maximum value, a scaling factor of between 3 and 5 might be considered. Our main concern here is fitting all the mirrored density plots on the same panel, and so we choose to align by maximum value (more generally, by range); after some iteration, I've chosen a scaling factor of 5, as the default tick marks on the secondary axis then align with the same gridlines on the visualisation.

```
# Q6
white %>%
  select(c('citric.acid', 'volatile.acidity', 'sulphates', 'chlorides')) %>%
  summary()
```

```
## citric.acid volatile.acidity sulphates chlorides
## Min. :0.0000 Min. :0.0800 Min. :0.2200 Min. :0.00900
## 1st Qu.:0.2700 1st Qu.:0.2100 1st Qu.:0.4100 1st Qu.:0.03600
## Median :0.3200 Median :0.2600 Median :0.4700 Median :0.04300
## Mean :0.3342 Mean :0.2782 Mean :0.4898 Mean :0.04577
## 3rd Qu.:0.3900 3rd Qu.:0.3200 3rd Qu.:0.5500 3rd Qu.:0.05000
## Max. :1.6600 Max. :1.1000 Max. :1.0800 Max. :0.34600
```

```
scalefactor <- 5
violinplots + scale_y_continuous(sec.axis=sec_axis(~./scalefactor))
```



7. Now return to your definition of `white_tidy2` and make the necessary change, such that the resulting `chlorides` density is appropriately represented by the secondary axis. Add labels to both vertical axes to clarify which variables they represent.

Although we've added a secondary axis, this is essentially only a 'dummy' axis, and the variables are all still plotted with respect to the primary axis. We should therefore scale the values in `chlorides`, and we can do this using `dplyr::mutate()`, as shown in the code below. As you can see, I have created a new tibble, `white_tidy_scaledchlorides`, rather than replacing the old one. I have done this (and explicitly referred to the scaling in the new tibble's name), as this change is being made for plotting purposes only; in any further analysis, I want to avoid accidentally using the scaled values of `chlorides`.

Finally, we can use the `name=` argument to `scale_y_continuous()` to add labels to the primary y-axis, and the `name=` argument to `sec_axis()` to add labels to the secondary y-axis. Note that we could alternatively use `colour` to link a specific density to a particular axis; I've included an example of how to code this below.

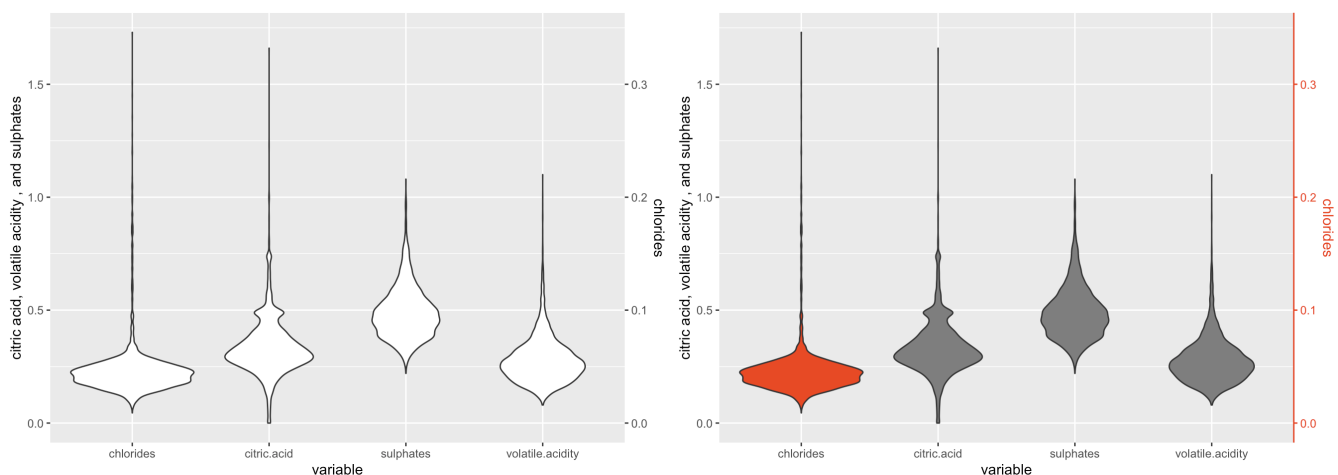
```
# Q7
white_tidy_scaledchlorides <- white %>%
  select(c('citric.acid', 'volatile.acidity', 'sulphates', 'chlorides')) %>%
  mutate(
    chlorides = chlorides * scalefactor
  ) %>%
  pivot_longer(cols=everything(), names_to='variable')

violinplots_final <- ggplot(data=white_tidy_scaledchlorides) +
  geom_violin(aes(x=variable, y=value)) +
  scale_y_continuous(
    name="citric acid, volatile acidity , and sulphates",
    sec.axis=sec_axis(~./scalefactor, name="chlorides")
  )

violinplots_final

violinplots_redaxis <- ggplot(data=white_tidy_scaledchlorides) +
  geom_violin(aes(x=variable, y=value,
    fill=factor(ifelse(variable=="chlorides", "axis2", "axis1"))
  )) +
  scale_y_continuous(
    name="citric acid, volatile acidity , and sulphates",
    sec.axis=sec_axis(~./scalefactor, name="chlorides")
  ) +
  scale_fill_manual(name = "variable", values=c("grey50", "red")) +
  theme(
    legend.position = "none",
    axis.line.y.right = element_line(color = "red"),
    axis.ticks.y.right = element_line(color = "red"),
    axis.text.y.right = element_text(color = "red"),
    axis.title.y.right = element_text(color = "red")
  )

violinplots_redaxis
```

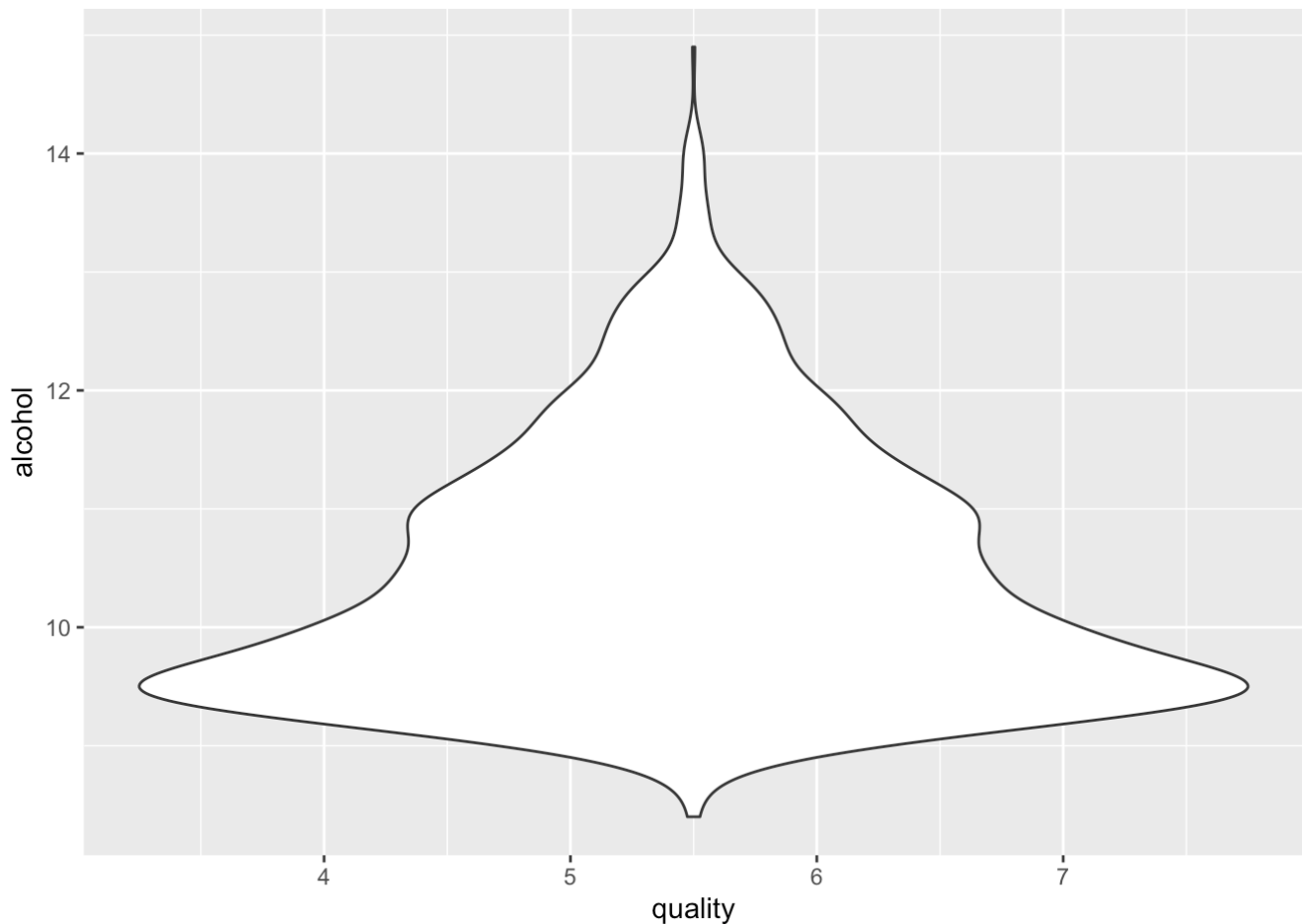


As we have seen, though these visualisations are conceptually straightforward, they can take a bit of work to construct. Nevertheless, they can be useful for comparing the univariate distributions within our dataset. We can also use these simple constructions to dive a bit deeper into the multivariate dependence structure in our dataset. Suppose we have a categorical variable (e.g. our red / white

variable, or quality). We can use the constructions above to display the conditional distribution of a numeric variable, when conditioning on these categories. We have already seen this technique in Week 4; we revisit it briefly here in the context of the Wine Quality dataset.

Suppose we wish to explore a potential link between the alcohol content of a wine, and its quality rating (as judged by expert oenologists, using blind tasting). In Week 4, we used `geom_boxplot()` , mapping both the `x` and `y` aesthetics to achieve a conditional boxplot. We will aim to use a similar approach here for creating a conditional violinplot visualisation.

```
condviolins <- ggplot(red) +
  geom_violin(aes(y=alcohol, x=quality))
condviolins
```



This has clearly not produced the required conditional violinplot; we would expect to see separate violinplot for each level of the categorical variable `red$quality` . We can force the split by also allocating `quality` to the `group` aesthetic, but this is a bit of a hack - we should find out why the above code didn't work.

By examining the variable *types* in our `red` dataset, we can see that `quality` is being treated as a numeric variable (as are all the others). This is causing us problems, as `ggplot()` needs to recognise this as a categorical variable, in order to automatically plot the densities by each category of `quality` .

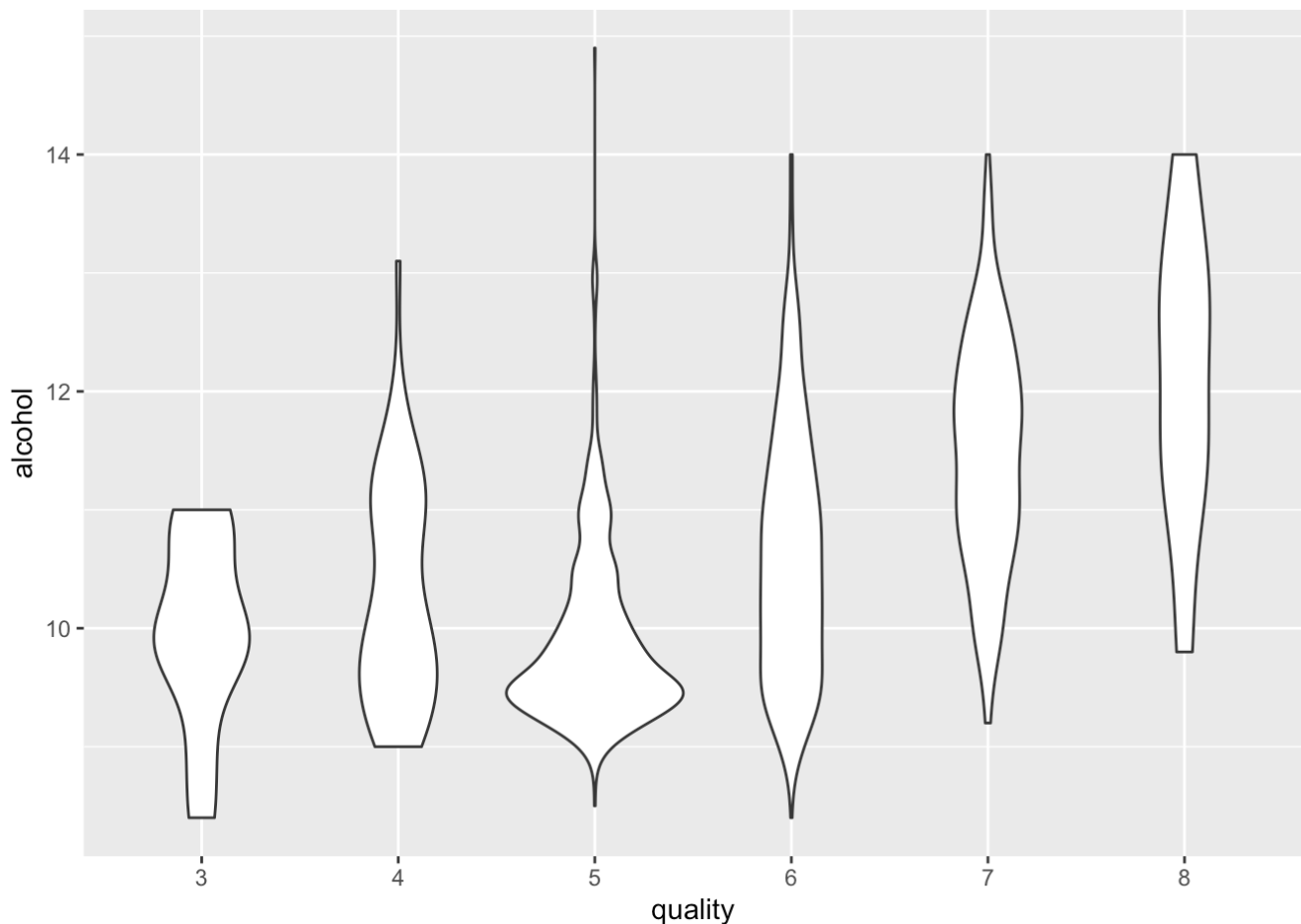
Exercises

8. Use `as.factor()` to change the type of `red$quality` . (*Explicitly changing a variable type like this is known as explicit coercion.*)
9. Now construct a violinplot visualisation that represents the distribution of `red$alcohol` , conditioned on the value of `red$quality` . Your visualisation should give six separate 'violin' density plots

10. Repeat this for the amalgamated wine dataset. Use the colour/fill aesthetics to further visualise the alcohol content when *jointly* conditioned on quality rating and wine colour.

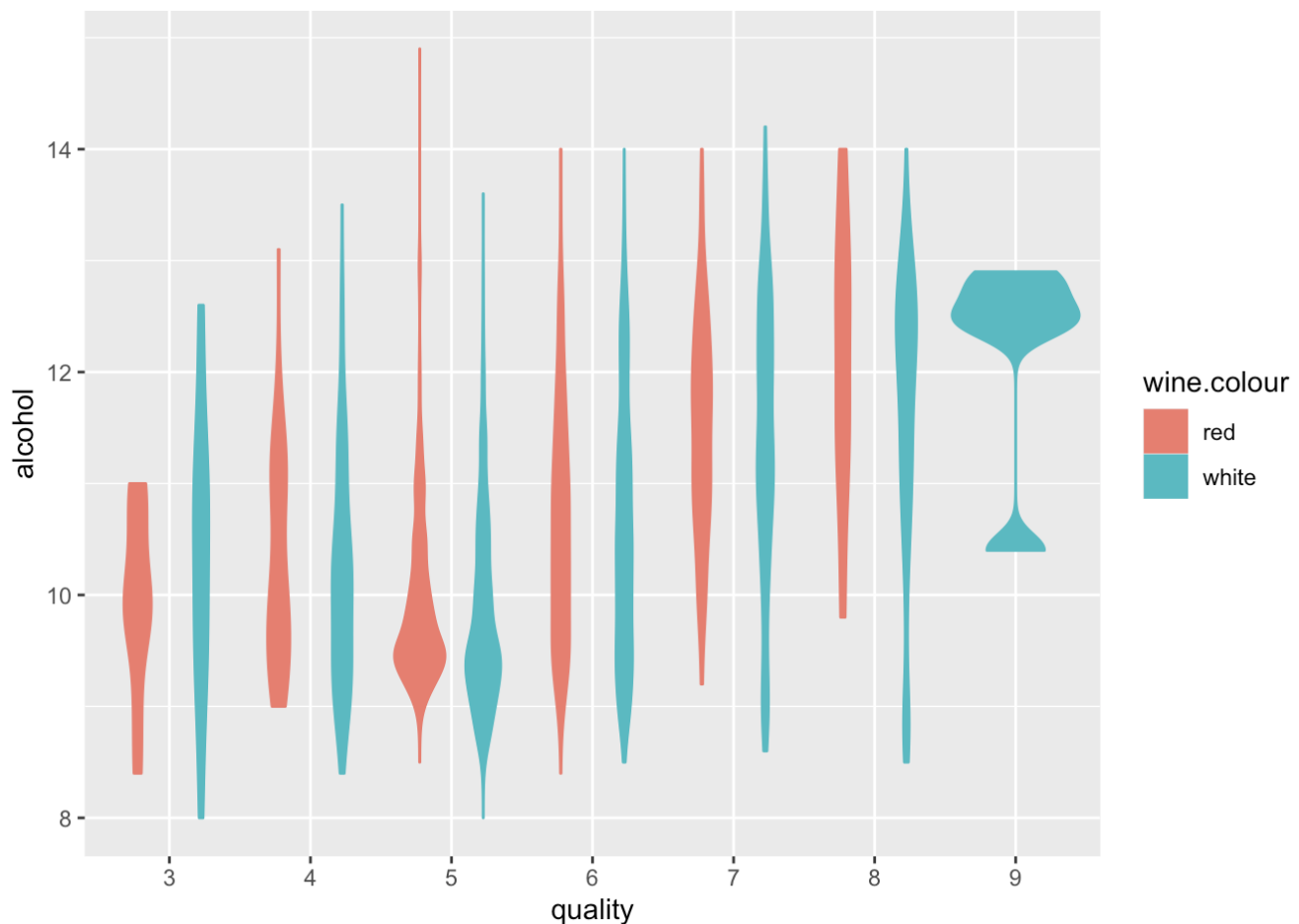
```
# Q8
red$quality <- as.factor(red$quality)

# Q9
condviolins <- ggplot(red) +
  geom_violin(aes(y=alcohol, x=quality))
condviolins
```



```
# Q10
wine$quality <- as.factor(wine$quality)
condviolins_2 <- ggplot(wine) +
  geom_violin(aes(y=alcohol, x=quality, colour=wine.colour, fill=wine.colour))

condviolins_2
```



The violin plots immediately above are an example of a bivariate visualisation where one variable is categorical. When both variables are numeric, we can return to another familiar visualisation - the scatterplot.

Bivariate Data Visualisation

We'll jump straight in, using a scatterplot to examine the bivariate relationship between the density of the wine and the residual sugar content (this is the natural sugar from the grape that remains in the wine after alcoholic fermentation).

Exercises

Using functions from the `ggplot2` library, and the `red` dataset...

11. Construct a scatterplot of `residual.sugar` against `density`. What initial patterns can you extract?

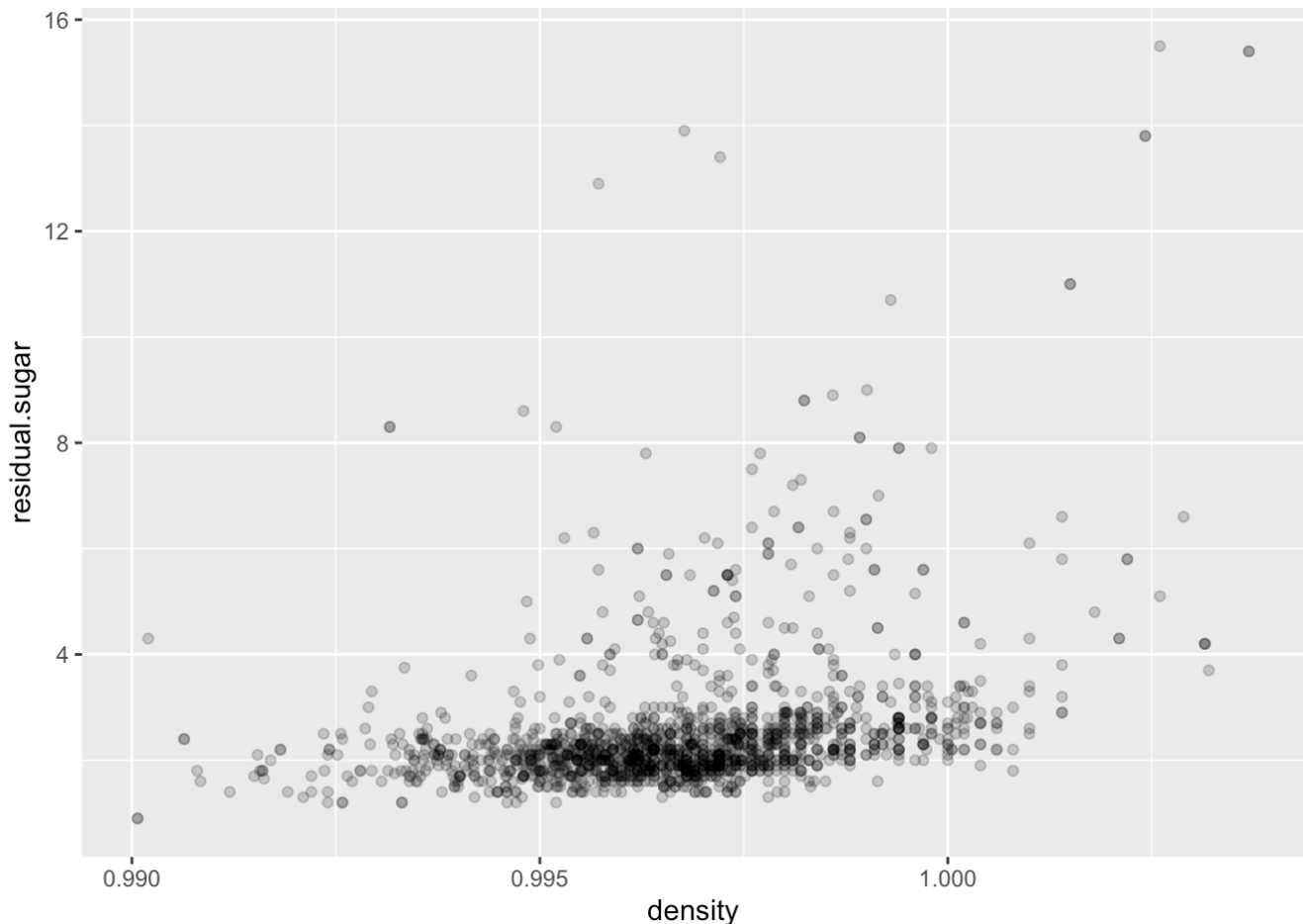
As discussed further below, this first scatterplot suggests a positive relationship between `density` and `residual.sugar`. We might also note that all values of `density` are between 0.99 and 1.05; we might consider whether it is appropriate and sensible to scale these values.

12. Try different values of `alpha` - what do you notice?

From varying `alpha`, we can see that there is a degree of overplotting; some of the points correspond to multiple observations. As we saw in Week 4, we can address this using either `geom_jitter()` (with appropriate specification of `height` and `width` parameters) or `geom_count()`.

```
# Q11, Q12
bv_scatter <- ggplot(data = red, mapping = aes(x=density, y=residual.sugar)) +
  geom_point(alpha=0.2)
# geom_jitter(alpha=0.2, width=0.0002, height=0.4)
# geom_count(alpha=0.2)

bv_scatter
```



A clear pattern emerges from this scatterplot, suggesting a positive relationship between `density` and `residual.sugar`. We can also see that there is a reasonable number of observations that have higher-than-average `residual.sugar` values. This is evidence that the marginal distribution of `residual.sugar` is skewed, though this is perhaps not *intuitively* clear from this bivariate visualisation. It would be interesting to examine the marginal distributions (particularly for `residual.sugar`) at the same time as examining the bivariate information. Can we combine this information on the same visualisation?

We're going to consider two alternative tools for this task - we're going to look at `geom_rug()` (which is part of the `ggplot2` library) and also at `ggMarginal()`, which can be found in the additional 'ggExtra' package.

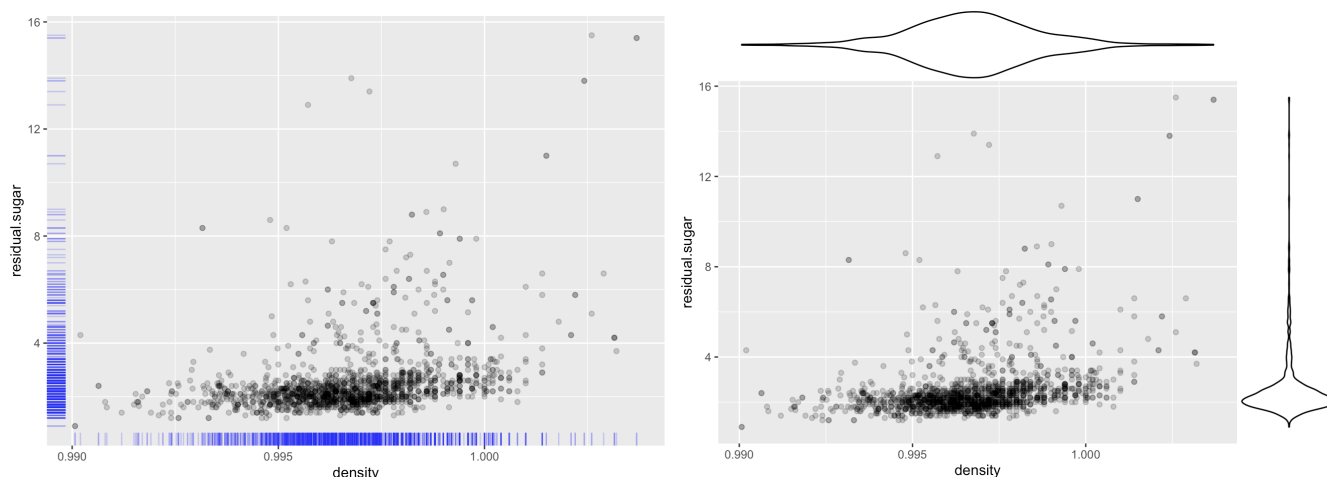
```

bv_joint_rug <- bv_scatter + geom_rug(col='blue', alpha=0.2)
bv_joint_marg <- ggExtra::ggMarginal(bv_scatter, type="violin")
# note that ggMarginal cannot be added as a layer in the usual ggplot syntax.

print(bv_joint_rug, newpage=TRUE)
print(bv_joint_marg, newpage=TRUE)

#bv_joint_rug
#bv_joint_marg

```



Both functions clarify the characteristics of the marginal distributions; it is clearer now that `residual.sugar` is positively skewed.

Exercises

13. Have a look at the parameters available for `ggExtra::ggMarginal()` (you can use the 'help' documentation, or take a look at this vignette (<https://cran.r-project.org/web/packages/ggExtra/vignettes/ggExtra.html>)). Try some of the alternative marginal distribution representations.

We can, again, incorporate additional variables into our plot to create a higher-order multivariate visualisation.

14. Repeat the scatterplot above for the same variables in the amalgamated `wine` dataset; use the colour aesthetic to encode the `wine.colour` variable.

Q14

```

bv_scatter_2 <- ggplot(data = wine, mapping = aes(x=density, y=residual.sugar, col
our=wine.colour)) +
  geom_point(alpha=0.2)

bv_joint_rug_2 <- bv_scatter_2 + geom_rug(alpha=0.2)
bv_joint_marg_2 <- ggExtra::ggMarginal(bv_scatter_2, type="violin", groupFill = TR
UE)

```

```

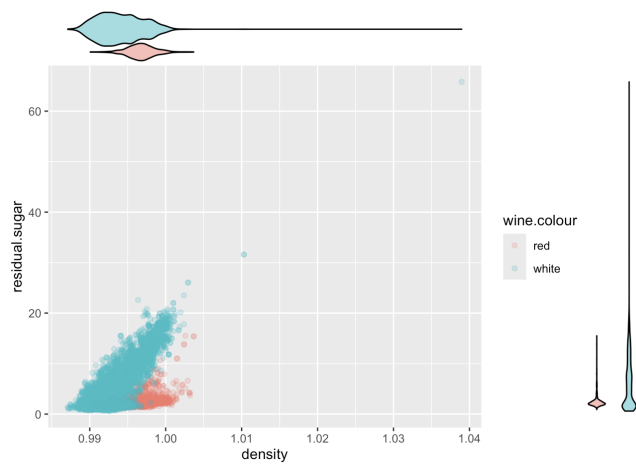
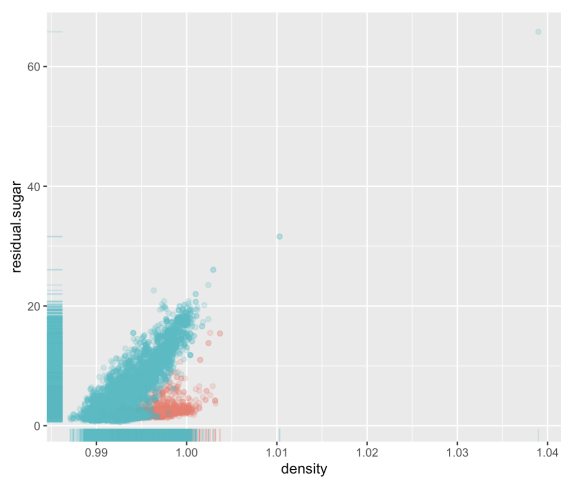
## Warning: `position_dodge()` requires non-overlapping x intervals.
## `position_dodge()` requires non-overlapping x intervals.

```

```

print(bv_joint_rug_2, newpage=TRUE)
print(bv_joint_marg_2, newpage=TRUE)

```



Visualising pairwise dependence

Moving from a bivariate analysis to a higher-order multivariate analysis, one visualisation option is to use a lattice approach, constructing bivariate visualisations for each pair of variables, and collating these in a panel plot.

At one point, the `ggplot2` library included a function `plotmatrix()` that implemented a pairwise scatterplot matrix. This has since been deprecated, however, in favour of the implementation provided by the `Gally` package. We demonstrate this implementation with the following exercises.

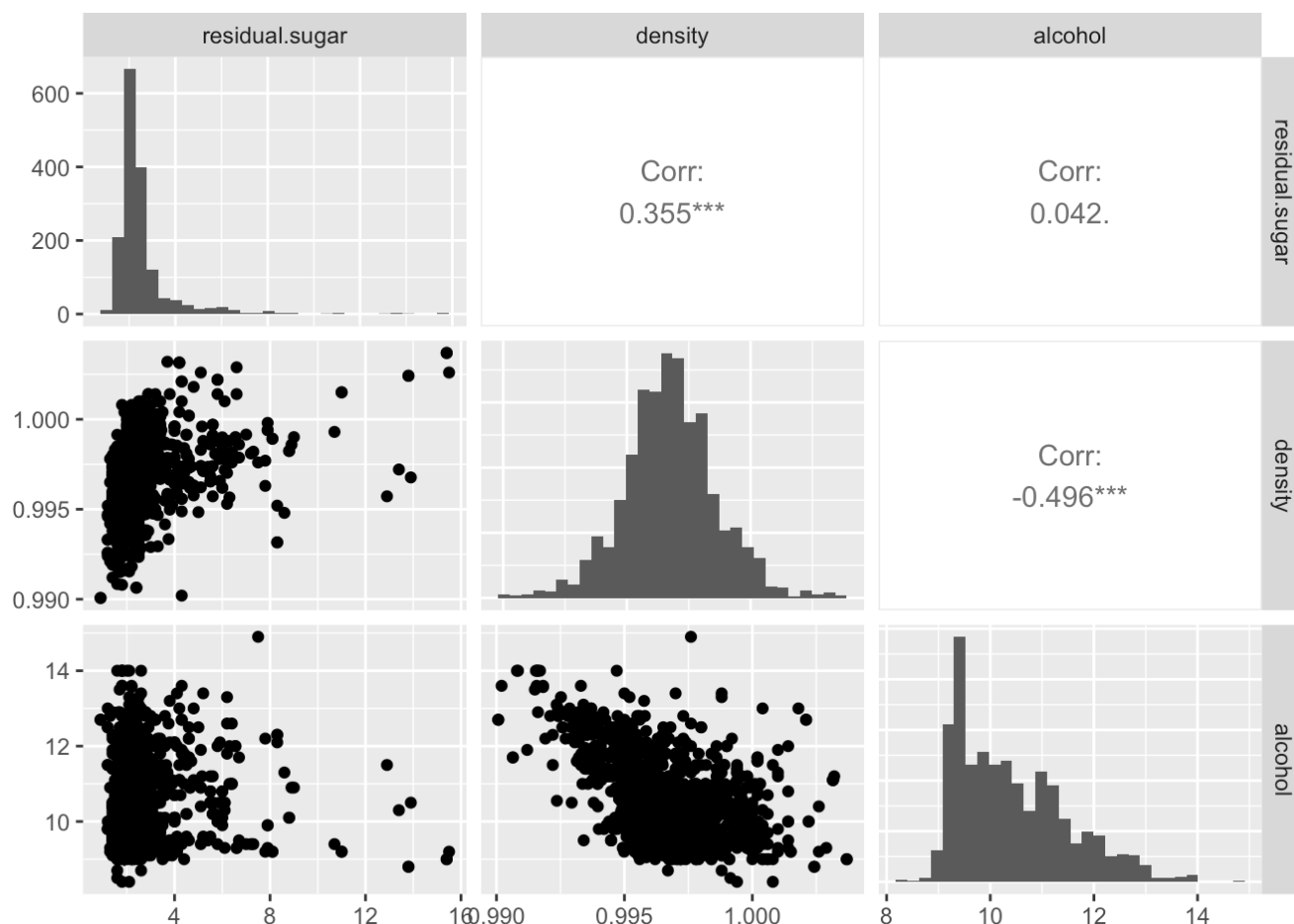
Exercises

15. Use the `ggpairs()` function to create a (trivariate) scatter plot matrix for the residual sugar, density and alcohol content in the red wines dataset.

```
# Q15
ggpairs(data = red,
        columns = c(4,8,11),
        diag = list(continuous = 'barDiag')
)
```

```
## Warning in geom_point(): All aesthetics have length 1, but the data has 9 rows.
## i Did you mean to use `annotate()`?
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



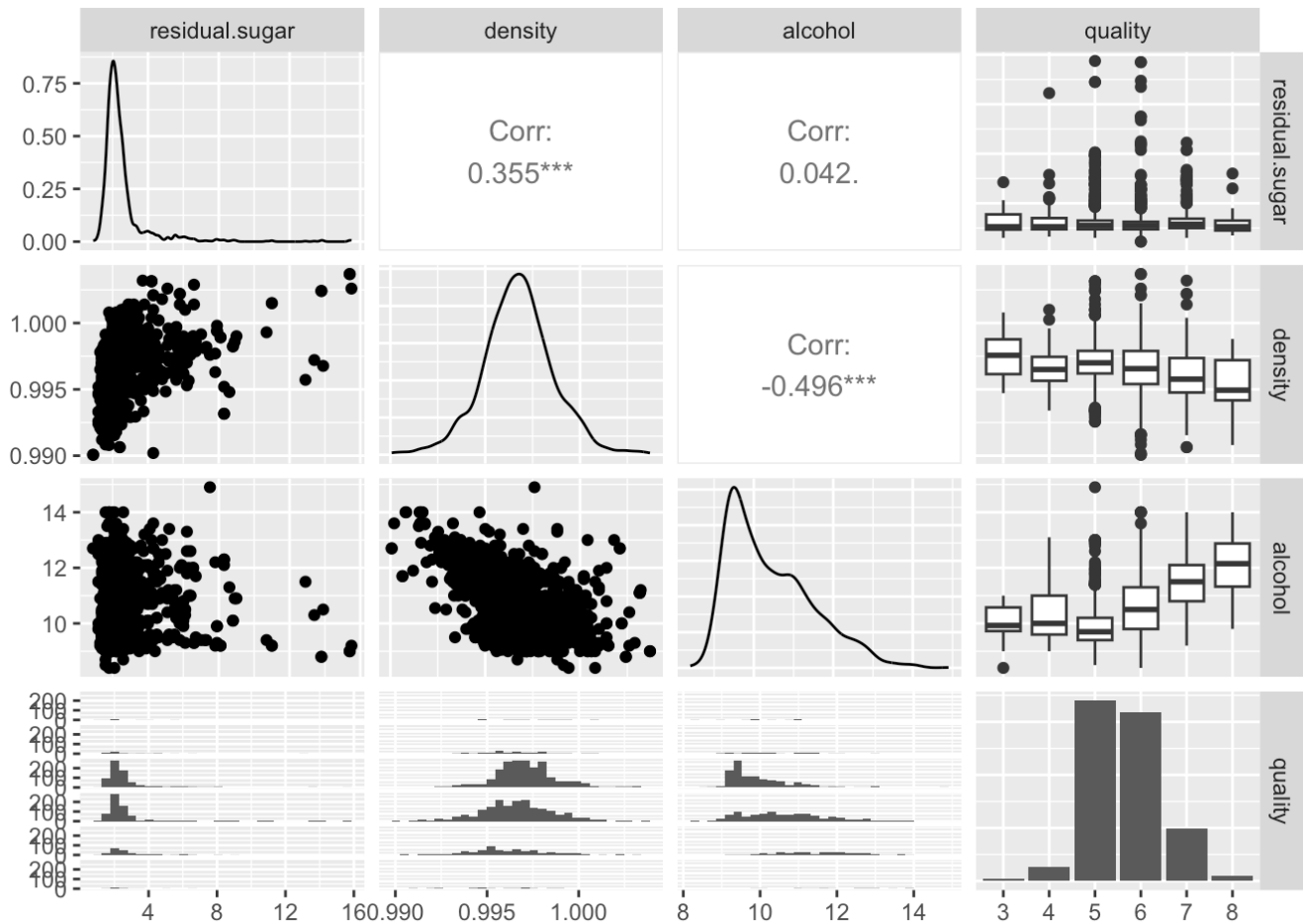
16. Examine the help documentation for `ggpairs()` and try out some different specifications for the `diag`, `upper` and `lower` arguments.
17. Extend your plot matrix to four variables, by including the categorical variable `quality`. If you haven't done so already, you will need to explicitly coerce this to a factor type.

As `quality` is a categorical variable, `ggpairs()` automatically changes its default visualisations for the corresponding variable pairs. Rather than a scatter plot, we get two lattice visualisations (containing histograms and boxplots respectively) to show us the distribution of each continuous variable, conditioned on `quality`.

```
# Q17
# red$quality <- as.factor(red$quality)
ggpairs(data = red,
        columns=c(4,8,11,12)
        )
```

```
## Warning in geom_point(): All aesthetics have length 1, but the data has 16 rows
.
## i Did you mean to use `annotate()`?
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

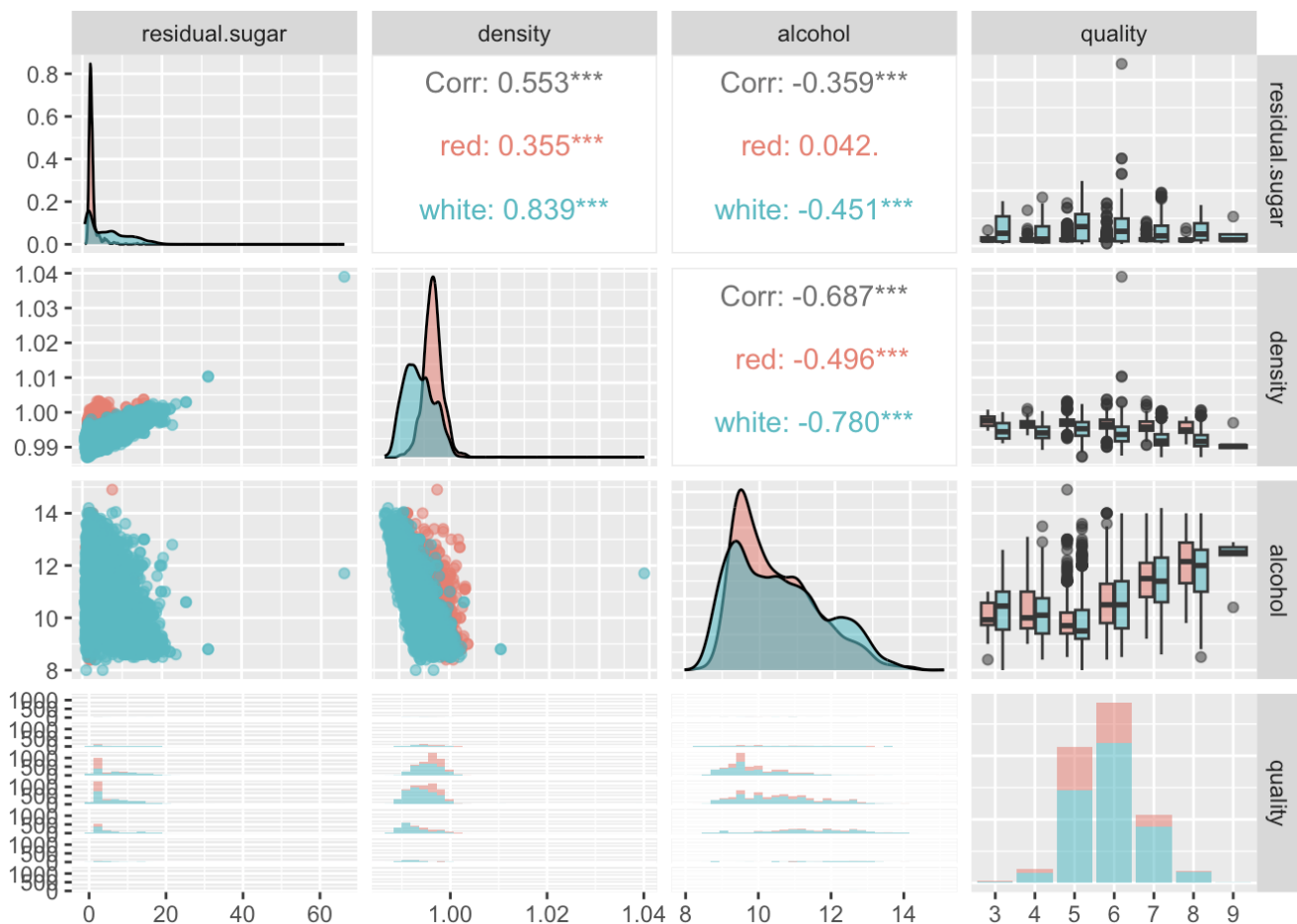


18. Finally, repeat this visualisation for the combined wine dataset, using the colour/fill aesthetics to encode the wine.colour variable.

```
# Q18
# wine$quality <- as.factor(wine$quality)
ggpairs(data = wine,
        columns=c(5,9,12,13),
        aes(colour=wine.colour, alpha=0.2)
        )
```

```
## Warning in geom_point(): All aesthetics have length 1, but the data has 16 rows
.
## i Did you mean to use `annotate()`?
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

Note the distinction between this and the ggplot technique of *faceting*. When using facets, we keep both axis variables fixed, and different panels correspond to distinct levels of a third, faceting variable; here, each panel within the upper or lower triangle corresponds to a distinct pair of axis variables.

We conclude this notebook by demonstrating a useful visual representation of a multivariate correlation structure, which is relatively straightforward to construct. For a multivariate dataset it is common to summarise the multivariate dependence structure using a (symmetric) matrix of pairwise correlation coefficients; we can visualise such a matrix using a 'heatmap'.

Exercises

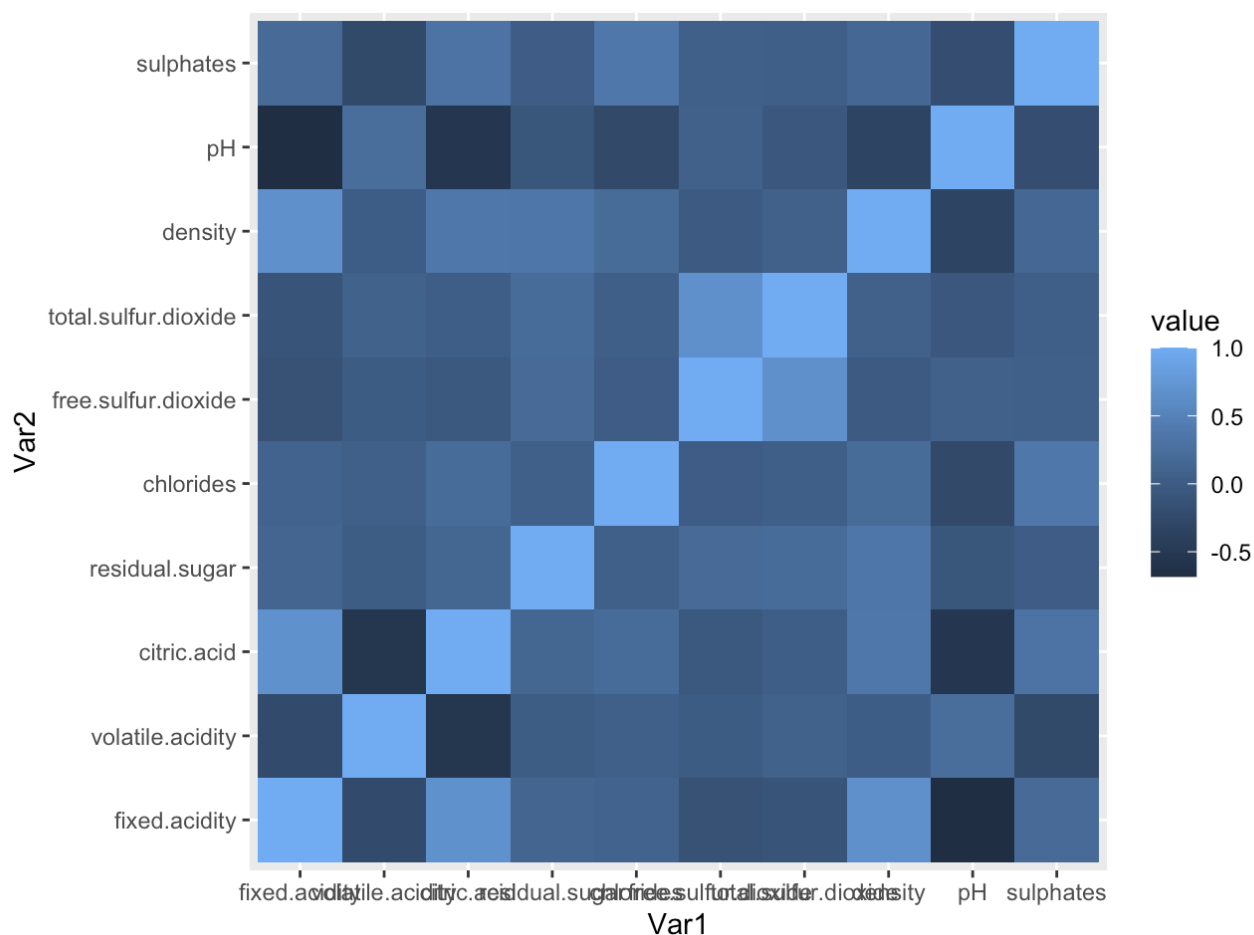
The approach we demonstrate in these exercises makes use of the `reshape2` library to convert the correlation matrix into a dataframe that can be interpreted by `ggplot()`.

19. Calculate the correlation matrix of the first 10 variables in the red wine dataset.
20. Use `reshape2::melt()` to 'melt' the resulting correlation matrix into a suitable dataframe.
21. Create a ggplot2 object to visualise the resulting dataframe using `geom_tile()`; you will need to consider the most appropriate variables to map the `x`, `y` and `fill` aesthetics to.

```
# Q19
corrmatrix <- cor(red[1:10])

# Q20
df_corrmatrix <- melt(corrmatrix)

# Q21
corr_heatmap <- ggplot(df_corrmatrix, aes(x=Var1, y=Var2, fill=value)) +
  geom_tile() + coord_fixed()
corr_heatmap
```



References

- Cortez, P., Cerdeira, A., Almeida, F., Matos, T. and Reis, J. (2009). Modeling wine preferences by data mining from physicochemical properties. *Decision Support Systems*, 47(4), pp. 547–543. Preprint available here (<http://www3.dsi.uminho.pt/pcortez/wine5.pdf>).